

CAM-Resident Behavioral Monitoring for LLMs:

Companion Appendices

Anonymized for review

DATE 2027 submission – companion to the main paper; sections here are cited from the main text as the red-team appendix (App. A) and

A. OBJECTIONS AND RESPONSES

The argument of this paper has standing objections. We state the ones we consider strongest, each with our response.

(1) The library may stay small: A probe trained on all domains jointly generalizes well in the same study that reports the non-transfer result [1], and a later analysis finds roughly 67% of top attention heads shared between sycophancy and factual lying, with mean-difference directions on the shared subspace transferring at AUROC 0.83 [2]. Deployed monitors are also already inexpensive [3]. If a compact shared basis covers a behavior suite, detection stays at small K and a GPU is sufficient for it. *Response:* That possibility does not reach the readout workloads, whose K is set by the vocabulary rather than by how many behaviors share a direction, and it is the readout end of Fig. 1 of the main paper that the hardware argument addresses. Our own measurements also cut against a compact shared basis. On the ten-domain grid, the full-precision ceiling itself transfers poorly off-diagonal (0.705, against 0.928 in-domain), so the transfer gap is in the representations rather than in any compression this paper introduces; and giving the encoding *more* capacity makes cross-domain transfer worse, not better – growing the code from 384 to 2,048 bits drops 23 of 90 off-diagonal cells by more than 0.05 AUROC (main paper, Sec. V-C). If transfer were limited by representational capacity, more bits should have helped; that they do not is consistent with behaviors wanting their own stored directions, i.e., with K growing. We have also built the combined row for our own banks: pooled over all ten domains’ training splits (11,278 rows), the per-domain AUROC averages 0.906 (thermometer, 384 b), 0.909 (ternary, 384 b), and 0.927 (full-precision ceiling), and sycophancy stays at 0.63 – pooling costs nothing against the per-domain banks, but it does not reproduce the combined probe’s 0.97 on sycophancy (main paper, Sec. V-B). **Item 2: our own combined-row numbers, per MN’s note that we should report them rather than argue around theirs.** That said, this is one model and one grid, and whether a shared basis suffices for a behavior suite remains an open empirical question; the analysis of this paper is conditional on the published trajectory continuing.

(2) Nobody searches a large library exhaustively: Approximate indexes visit a small fraction of a bank and retrieve most true neighbors, at sub-millisecond query times over 10^8 vectors on a GPU [4]–[6]. *Response:* A monitored

query stream is adversarial rather than independently drawn, and index structures with adversarial-query guarantees have been shown to cost a $\sqrt{Q}$ space factor [7]; an exhaustive scan has no approximation error for an attacker to exploit. Neither observation settles the comparison, and the recall-under-adversarial-queries measurement is item (2) of the main paper’s measurement plan (Sec. VII-F).

(3) A datacenter GPU amortizes the bank read: At batch 64 the 2.1 GB per layer per token becomes roughly 34 MB per request, so a loaded node pays far less per request than the batch-1 arithmetic suggests. *Response:* Correct, and the cost model of the main paper’s Sec. VII-F calls for the baseline at batch 1 *and* batch 64 for this reason. A phone or a robot answering one request at a time has nothing to amortize against, which is why the edge crossings arrive one to two orders of magnitude earlier in K ; and a co-resident monitor contends with live inference for cache and bandwidth, a term not yet in anyone’s numbers.

(4) The compression is validated for detection, not for action: The measured fidelity covers stored-signature *matching*. A monitor that corrects behavior by steering would need full-precision, steering-grade directions, which the compressed bank does not carry. *Response:* Agreed; that is why the main paper’s Sec. VII-B keeps the raw-bank arithmetic alongside the compressed one, and why detection – flag and escalate – is the claim this paper makes.

(5) The upper rungs are methods, not deployments: Only $K = 1$ runs in production; the 10^2 – 10^3 rungs are measurements and an incident report, and the 10^5 rung is a published readout method nobody has deployed as a monitor. *Response:* Stated as such throughout (the arrow in Fig. 1 of the main paper is not a forecast), and the argument does not require the top rung: the libraries already in hand – 520–1,000 rows for four observed behaviors – sit near the on-chip capacity cliff of an edge part (main paper, Sec. VII-B), so the architectural question arrives before whole-vocabulary readout does.

B. THE BANDWIDTH TAX, AND EXPERIMENTS TO MEASURE IT

[parked here per your request (“fold this into a new appendix for now about things we might want to try”); it can move into Sec. VII or get cut once you have read it]

What would it do to inference if monitoring consumed 10–20% of an H100’s memory bandwidth? To first order the

tax passes through one-for-one, because decode is memory-bandwidth-bound across essentially all practical serving configurations: throughput falls by the stolen share, and inter-token latency rises as $1/(1-x)$, i.e., +11% at a 10% tax and +25% at 20%. In numbers, a 70B model at fp8 is roughly 70 GB of weights, so batch-1 decode on one H100 has a streaming floor near 48 tokens/s – about 43 under a 10% tax and 38 under 20%. At batch 32 with 8K contexts the weights amortize but each stream drags its own key-value (KV) cache (roughly 1.3 GB per stream for a grouped-query 70B at fp8), so a decode step moves ~ 112 GB and the node serves ~ 950 aggregate tokens/s – again falling in proportion, to ~ 760 at a 20% tax. Notably, batching does not remove the tax, since KV traffic keeps decode bandwidth-bound at every practical batch; prefill is the exception, being compute-bound, so time-to-first-token moves little.¹

Three second-order effects could make the same tax cost more than its face value. **(1) Queueing amplification:** a serving node runs at some utilization against a latency target; removing 20% of capacity at a fixed arrival rate moves utilization of 0.75 to roughly 0.94, at which point tail latency grows several-fold and goodput-under-SLO falls much faster than the bandwidth did. The unit an operator would reason in is therefore fleet size: a 10–20% bandwidth tax is 10–25% more GPUs to hold the same service level. **(2) Contention is not a clean partition:** the monitor’s bank scan is a streaming read that also evicts the 50 MB L2 that attention and weight kernels reuse, and DRAM scheduler and row-buffer interference between co-running streams can push observed degradation past the raw bandwidth share. Current parts offer no fine-grained bandwidth QoS; MIG isolates the monitor, but a MIG slice is itself a dedicated capacity loss. **(3) Energy:** the tax is approximately proportional in HBM access energy, which at realistic K , layer counts, and token rates is the kilowatt-scale arithmetic of the main paper’s Sec. VII-E.

Each of these maps to a short measurement, none requiring training: **(1)** a *bandwidth-thief* microbenchmark – co-run a tuned streaming kernel (or the monitoring GEMV itself, over a $K \times d$ bank) against a serving stack (e.g., vLLM) running a 70B on one H100; sweep the thief at 0/5/10/20/40% of peak bandwidth; record decode tokens/s, inter-token latency at the median and tail, and time-to-first-token, at batch 1/8/32/64 and 1K/8K/32K contexts. The slope of throughput against stolen bandwidth is the pass-through factor: 1.0 confirms the first-order story, larger measures the contention overshoot. **(2)** the same thief with L2 access-policy controls (persistence windows, bypass hints), separating cache pollution from bandwidth. **(3)** the thief under MPS co-residence against a MIG slice, pricing the isolation options. **(4)** the monitor end-to-end: a per-token FAISS/cuBLAS scan at $K = 10^4$ – 10^5 , $d = 8,192$ hooked into the serving loop, with the measured tokens/s delta compared against the analytic $KdbR$ of Eq. (2) of the main paper. **(5)** a fixed-arrival-rate load test measuring goodput at

an inter-token-latency SLO with and without a 10/20% thief, which is the queueing-amplification number. Experiment (1) is the highest value per day and pairs naturally with the tuned-GPU-baseline item of the main paper’s Sec. VII-F; a Jetson-class repeat of (1) at batch 1 covers the edge case, where the first-order story should hold almost exactly.

REFERENCES

- [1] Z. J. Ying, S. Ravfogel, N. Kriesegkorte, and P. Hase, “The truthfulness spectrum hypothesis,” *arXiv preprint arXiv:2602.20273*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.20273>
- [2] M. Pandey, “Llms know they’re wrong and agree anyway: The shared sycophancy-lying circuit,” *arXiv preprint arXiv:2604.19117*, 2026, single-author preprint. [Online]. Available: <https://arxiv.org/abs/2604.19117>
- [3] Anthropic Alignment Science, “Cheap monitors,” Anthropic alignment blog, 2025. [Online]. Available: <https://alignment.anthropic.com/2025/cheap-monitors/>
- [4] P. Indyk and R. Motwani, “Approximate nearest neighbors: Towards removing the curse of dimensionality,” in *ACM Symposium on Theory of Computing (STOC)*, 1998, pp. 604–613.
- [5] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [6] Meta Engineering and NVIDIA, “Accelerating gpu indexes in faiss with nvidia cuvs,” Meta Engineering blog, 2025. [Online]. Available: <https://engineering.fb.com/2025/05/08/data-infrastructure/accelerating-gpu-indexes-in-faiss-with-nvidia-cuvs/>
- [7] A. Andoni, T. Haris, E. Kelman, and K. Onak, “Efficient algorithms for adversarially robust approximate nearest neighbor search,” *arXiv preprint arXiv:2601.00272*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.00272>

¹Arithmetic on public specifications, not measurements. Achievable HBM bandwidth is typically 85–90% of the 3.35 TB s^{-1} figure, which shifts the absolute numbers but not the proportional argument.